МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

З лабораторної роботи №1

з дисципліни «Штучний інтелект у програмній інженерії»

Виконала:

ст. гр. ПЗПІ-23-4
Федькова К. О.

Перевірив:

Доцент кафедри ПІ
Каук В. І.

Харків 2026

Побудова AI-агента: власний цикл Observe, Reason, Act, Evaluate і реалізація на Google ADK

Мета

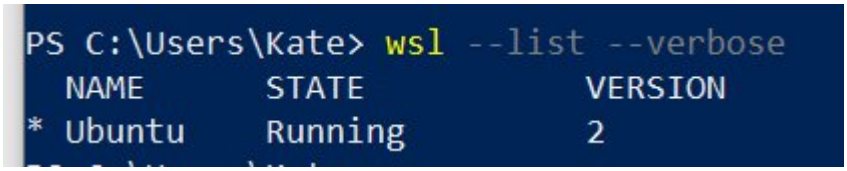
Налаштувати відтворюване середовище розроблення AI-агентів; побудувати на Python власного мінімального агента з циклом Observe, Reason, Act, Evaluate без agent framework; відтворити того самого агента на Google ADK з власним інструментом; експериментально порівняти одиничний виклик LLM, власного агента і ADK-агента на п’яти типах запитів; оформити паспорт агента.

Предметна область і зв’язок із кваліфікаційною роботою

Обрана предметна область - запахи коду. Темою кваліфікаційної роботи є соціальна книжкова платформа. Під час розробки проекту дуже важливо помічати і виправляти запахи коду, щоб зробити код більш простим для підтримування та доповнення.

Конфігурація середовища

Команда	Результат
wsl --list --verbose (PowerShell)	Ubuntu, VERSION 2
git --version	git version 2.43.0
uv --version	uv 0.13.5
uv python list	cpython-3.12
agents-cli --version	agents-cli, version 1.11.0
git check-ignore -v .env	.gitignore:1:.env .env



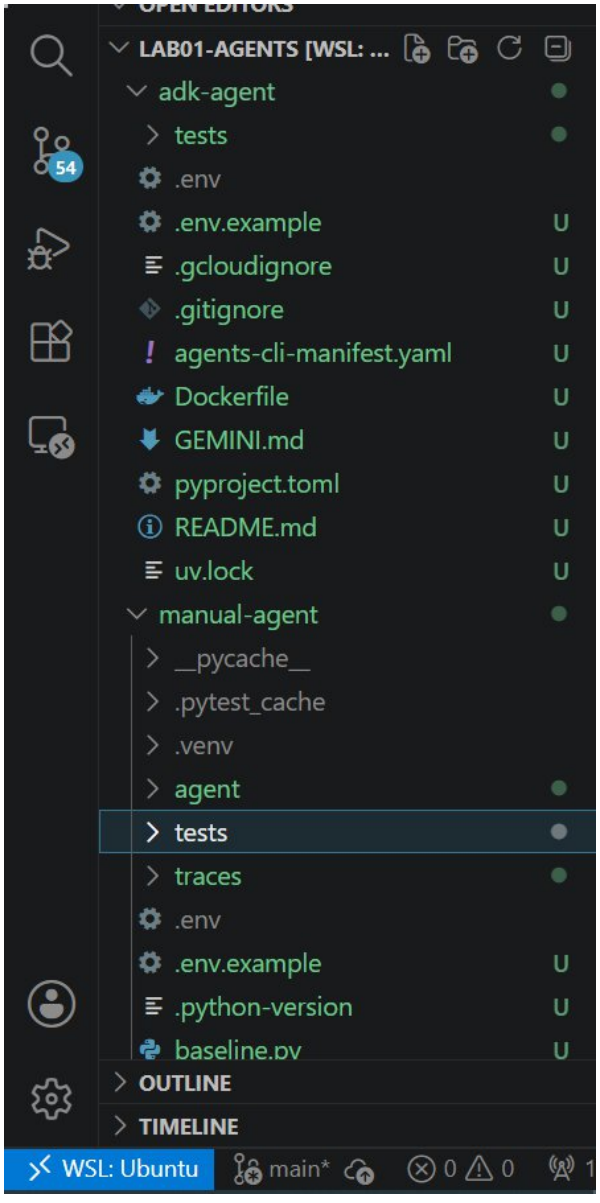
```
kate@Lenovo:~/ai-agents/lab01-agents/adk-agent$ git --version
git version 2.43.0
```

```
kate@Lenovo:~/ai-agents/lab01-agents/adk-agent$ uv --version
uv 0.13.5 (x86_64-unknown-linux-gnu)
```

```
cpython-3.12.15-linux-x86_64-gnu
cpython-3.12-linux-x86_64-gnu/bin/python3.12
cpython-3.12.15-linux-x86_64-gnu
thon3.12
cpython-3.12.3-linux-x86_64-gnu
cpython-3.12.3-linux-x86_64-gnu
```

```
kate@Lenovo:~/ai-agents/lab01-agents/adk-agent$ agents-cli --version
agents-cli, version 1.11.0
```

```
kate@Lenovo:~/ai-agents/lab01-agents$ git check-ignore -v .env
.gitignore:1:.env       .env
```



Архітектурна схема агента

USER → GOAL → STATE → OBSERVE → LLM → DECISION → VALIDATOR → DISPATCHER → ENVIRONMENT → OBSERVATION → STATE UPDATE → STOP

Середовище:

Об’єкти: duplicated-code, long-method, large-class, feature-envy, shotgun-surgery, data-clumps, primitive-obsession

Джерела фактів:

- Martin Fowler, "Refactoring: Improving the Design of Existing Code", 2nd Edition, 2018;
- Martin Fowler, "Refactoring" - каталог Code Smells: https://refactoring.com/catalog/;
- Joshua Kerievsky, "Refactoring to Patterns", 2004.

Код дії lookup:

```
def lookup_code_smell(name: str) -> str:
    """
    Returns reference information about a specific Code Smell.

    Supports the names: duplicated-code, long-method, large-class,
    feature-envy, shotgun-surgery, data-clumps, and
    primitive-obsession.

    Args:
        name: The name of the code smell, e.g., "Long Method".

    Returns:
        The identifier and description of the Code Smell, or a NOT_FOUND
message.
    """
    key = normalize_name(name)

    if key not in CODE_SMELLS:
        return f"{NOT_FOUND}: no verified information about '{name}'."

    return f"{key}: {CODE_SMELLS[key]}"
```

AgentState, контракт рішення, інструкція компонента рішень

```
from dataclasses import dataclass, field
from typing import Any

@dataclass
class AgentState:
    """A snapshot of the agent's execution state at the current step."""

    goal: str
    observations: list[str] = field(default_factory=list)
    trace: list[dict[str, Any]] = field(default_factory=list)
    step: int = 0
    llm_calls: int = 0
    finished: bool = False
    final_answer: str | None = None
    stop_reason: str | None = None
```

Контракт рішення:

```
{
    "action": "lookup_code_smell",
     "arguments": {"name": "Long method"},
     "decision_summary": "Verified data about Long Method is needed."
}
{
      "action": "finish",
      "arguments": {"answer": "Long method is …"},
     "decision_summary": "Method is described in observations."
}

AGENT_INSTRUCTION = """
You are the decision component of a small educational AI agent.
You do not chat. On every call you choose exactly one action.

Available actions:
1. lookup_code_smell
   Use it when verified information about a code smell is needed.
```

```
   Arguments: {"name": string}  e.g. "duplicated code", "long method",
"large class"
2. finish
   Use it when the verified observations are enough to reach the goal,
   or when the environment reported that the information does not exist.
   Arguments: {"answer": string}   final answer in Ukrainian

Return valid JSON only:
{
    "action": "lookup_code_smell | finish",
    "arguments": {...},
    "decision_summary": "one short sentence: why this action"
}

Rules:
- Use only facts from VERIFIED OBSERVATIONS in the final answer.
- If an observation starts with NOT_FOUND, say the information is
missing; do not invent it.
- If an observation starts with ERROR, fix your previous decision.
- Do not repeat a lookup that is already present in the observations.
- decision_summary is one sentence; do not reveal hidden chain-of-
thought.
"""
```

Валідатор і диспетчер: код і пояснення

Модель є ймовірнісним компонентом і час від часу повертає дію, якої немає, пропускає аргумент або передає число замість рядка. Валідатор перевіряє рішення до виконання за білим списком дій в який входять: lookup_code_smell, finish. Також валідатор перевіряє, чи чи arguments це словник, чи не пропущені параметри та чи вони не порожні.

```
"""Guardrail: the model proposes, the program decides."""

from .errors import DecisionError

# Whitelist of actions and mandatory arguments (action space)
ALLOWED_ACTIONS: dict[str, set[str]] = {
```

```
    "lookup_code_smell": {"name"},
    "finish": {"answer"},
}


def validate_decision(decision: dict) -> None:
    action = decision.get("action")
    if action not in ALLOWED_ACTIONS:
        raise DecisionError(f"unknown action: {action!r}")

    arguments = decision.get("arguments")
    if not isinstance(arguments, dict):
        raise DecisionError("arguments must be a JSON object")

    missing = ALLOWED_ACTIONS[action] - arguments.keys()
    if missing:
        raise DecisionError(f"missing arguments for {action}:
{sorted(missing)}")

    for key in ALLOWED_ACTIONS[action]:
        value = arguments[key]
        if not isinstance(value, str) or not value.strip():
            raise DecisionError(f"argument {key!r} must be a non-empty
string")
```

Диспетчер перетворює перевірене рішення на виклик Python функції. Він перевіряє дію, викликає відповідну функцію та оновлює стан агенту

```
def execute_action(decision: dict, state: AgentState) -> str | None:
    """Dispatcher: converts a verified solution into a Python function
call."""
    action = decision["action"]
    args = decision["arguments"]
    if action == "lookup_code_smell":
        return lookup_code_smell(args["name"])
    if action == "finish":
        state.final_answer = args["answer"]
```

```
            state.finished = True
            state.stop_reason = "finish"
            return None
    raise DecisionError(f"no handler for action {action!r}")
```

Цикл агента, умови зупинки, обґрунтування MAX_STEPS

Робота агента реалізована у вигляді ітераційного циклу, у якому на кожному кроці LLM аналізує поточний стан, формує рішення щодо наступної дії, після чого система перевіряє та виконує це рішення. Результат виконання зберігається у вигляді спостереження, яке використовується моделю під час наступної ітерації.

На початку роботи створюється об’єкт AgentState, що містить мету агента, історію спостережень, поточний номер кроку, кількість викликів LLM, трасування та фінальну відповідь. Цикл продовжується, доки значення state.finished не стане істинним.

Перед початком кожного нового кроку перевіряється обмеження max_steps. Якщо кількість виконаних кроків досягла встановленого ліміту, агент завершує роботу, встановлює причину зупинки max_steps і формує повідомлення про вичерпання ліміту без отримання фінальної відповіді. Перевірка виконується до наступного виклику LLM, що дозволяє уникнути зайвих запитів і контролювати використання обчислювальних ресурсів.

Якщо ліміт не вичерпано, лічильник кроків збільшується, після чого функція decide() отримує мету, накопичені спостереження, номер поточного кроку та максимальну кількість кроків. Отримане рішення перевіряється функцією validate_decision(), а за умови успішної перевірки передається до execute_action() для виконання відповідної дії.

У разі виникнення DecisionError помилка перетворюється на текстове спостереження та додається до історії виконання. Це дозволяє моделі врахувати попередню помилку під час формування наступного рішення. Для кожного кроку також фіксуються виконана дія, її аргументи, опис рішення, результат і

тривалість виконання. Ці дані зберігаються у трасі state.trace та використовуються для аналізу поведінки агента.

Основними умовами зупинки є досягнення стану завершення state.finished та вичерпання встановленого ліміту кроків. Значення MAX_STEPS = 8, бо всього наведено в оточенні сім code smells та ще один крок для підбиття підсумків, якщо користувачу потрібно. Це забезпечує контрольованість виконання агента та запобігає нескінченному повторенню ітерацій.

```
def run_agent(goal: str, decide: DecideFn = get_decision,
              max_steps: int = MAX_STEPS) -> AgentState:
    state = AgentState(goal=goal)

    while not state.finished:
        # EVALUATE (бюджет): ліміт перевіряють до нового виклику LLM
        if state.step >= max_steps:
            state.finished = True
            state.stop_reason = "max_steps"
            state.final_answer = (
                f"Агент зупинився: вичерпано ліміт кроків ({max_steps})
"
                "без фінальної відповіді."
            )
            break

        state.step += 1
        record: dict = {"step": state.step}
        decision: dict | None = None
        started = time.perf_counter()
        try:
            state.llm_calls += 1                          # OBSERVE +
REASON
            decision = decide(state.goal, state.observations,
state.step, max_steps)
            validate_decision(decision)                   # VALIDATE
            observation = execute_action(decision, state)   # ACT
            record.update(action=decision["action"],
```

```
                            arguments=decision["arguments"],
                            decision_summary=decision.get("decision_summar
y", ""))
        except DecisionError as exc:
            # Помилка стає спостереженням, і модель отримує шанс
виправитися
            observation = f"ERROR: {exc}"
            record.update(action="rejected", arguments=None,
                            decision_summary="", rejected=decision)

        record["observation"] = observation
        record["seconds"] = round(time.perf_counter() - started, 2)
        state.trace.append(record)
        if observation is not None:
            state.observations.append(observation)
        print_step(record)

    return state
```

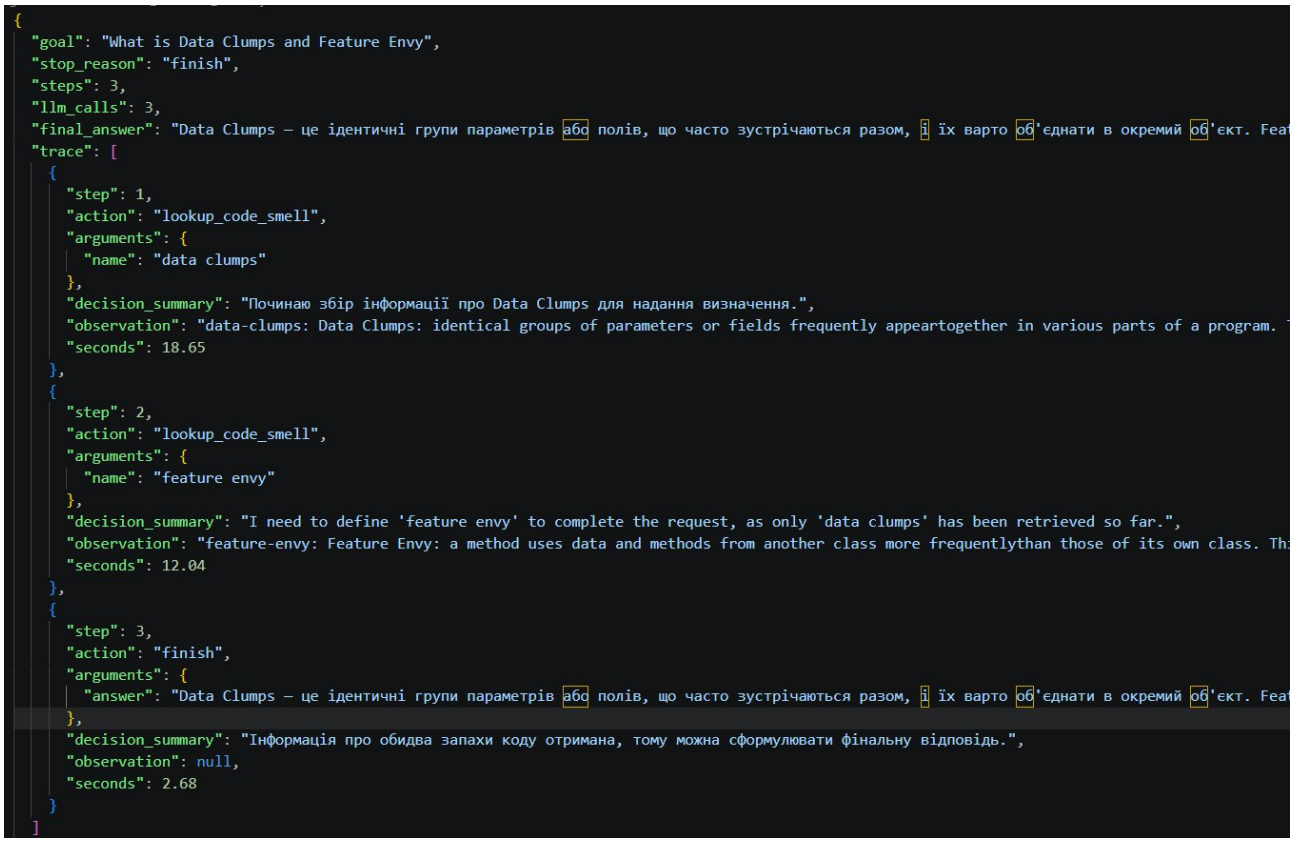
Приклад багатокрокового трасування з поясненням кожного кроку

На рисунку наведено результат роботи агента для запиту «What is Data Clumps and Feature Envy». Виконання складається з трьох кроків. На першому кроці агент викликає дію lookup_code_smell з аргументом data clumps для отримання інформації про відповідний code smell. Результат пошуку зберігається у полі observation і містить визначення Data Clumps. На другому кроці агент виконує аналогічний пошук для feature envy.

На третьому кроці агент обирає дію finish і передає сформовану відповідь у полі arguments. У короткому описі рішення (decision_summary) зазначено, що інформацію про обидва типи code smells отримано, тому можна сформувати фінальну відповідь. Поле observation на цьому кроці дорівнює null, оскільки додаткове спостереження після завершення не потрібне.

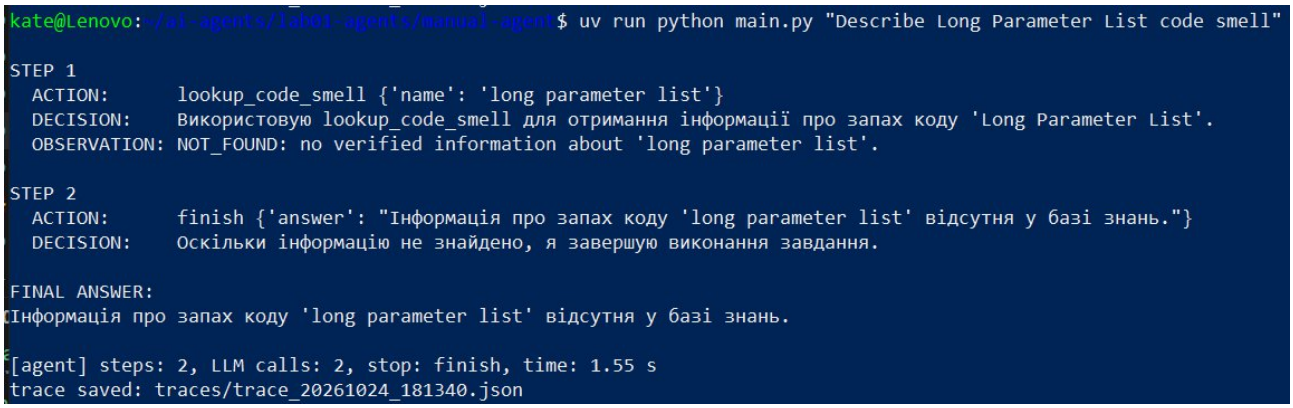
Загалом агент виконав три кроки та здійснив три виклики LLM. Загальний час виконання, розрахований за значеннями поля seconds, становить приблизно

33,37 секунди. Причиною зупинки є finish, що свідчить про штатне завершення роботи, а не про вичерпання ліміту кроків.



```
{
  "goal": "What is Data Clumps and Feature Envy",
  "stop_reason": "finish",
  "steps": 3,
  "llm_calls": 3,
  "final_answer": "Data Clumps – це ідентичні групи параметрів або полів, що часто зустрічаються разом, і їх варто об'єднати в окремий об'єкт. Feat
  "trace": [
    {
      "step": 1,
      "action": "lookup_code_smell",
      "arguments": {
        "name": "data clumps"
      },
      "decision_summary": "Починаю збір інформації про Data Clumps для надання визначення.",
      "observation": "data-clumps: Data Clumps: identical groups of parameters or fields frequently appeartogether in various parts of a program.
      "seconds": 18.65
    },
    {
      "step": 2,
      "action": "lookup_code_smell",
      "arguments": {
        "name": "feature envy"
      },
      "decision_summary": "I need to define 'feature envy' to complete the request, as only 'data clumps' has been retrieved so far.",
      "observation": "feature-envy: Feature Envy: a method uses data and methods from another class more frequentlythan those of its own class. Thi
      "seconds": 12.04
    },
    {
      "step": 3,
      "action": "finish",
      "arguments": {
        "answer": "Data Clumps – це ідентичні групи параметрів або полів, що часто зустрічаються разом, і їх варто об'єднати в окремий об'єкт. Feat
      },
      "decision_summary": "Інформація про обидва запахи коду отримана, тому можна сформулювати фінальну відповідь.",
      "observation": null,
      "seconds": 2.68
    }
  ]
```


Обробка двох відмов, зокрема зупинка за MAX_STEPS



```
kate@Lenovo:~/ai-agents/lab01-agents/manual-agent$ uv run python main.py "Describe Long Parameter List code smell"

STEP 1
  ACTION:      lookup_code_smell {'name': 'long parameter list'}
  DECISION:    Використовую lookup_code_smell для отримання інформації про запах коду 'Long Parameter List'.
  OBSERVATION: NOT_FOUND: no verified information about 'long parameter list'.

STEP 2
  ACTION:      finish {'answer': "Інформація про запах коду 'long parameter list' відсутня у базі знань."}
  DECISION:    Оскільки інформацію не знайдено, я завершую виконання завдання.

FINAL ANSWER:
Інформація про запах коду 'long parameter list' відсутня у базі знань.

[agent] steps: 2, LLM calls: 2, stop: finish, time: 1.55 s
trace saved: traces/trace_20261024_181340.json
```

```
kate@Lenovo:~/ai-agents/lab01-agents/manual-agent$ uv run python main.py "Compare Long Method and Data Clumps"

STEP 1
  ACTION:      lookup_code_smell {'name': 'long method'}
  DECISION:    Починаю збір інформації про перший код-смелл (Long Method) для подальшого порівняння.
  OBSERVATION: long-method: Long Method: a method contains too much logic and performs several distinctoperations, makin
g it harder to read, test, and modify. Refactoring approach:Extract Function or Replace Temp with Query.

STEP 2
  ACTION:      lookup_code_smell {'name': 'data clumps'}
  DECISION:    I need to retrieve information about 'data clumps' to complete the comparison with 'long method'.
  OBSERVATION: data-clumps: Data Clumps: identical groups of parameters or fields frequently appeartogether in various p
arts of a program. This may indicate that the data shouldbe combined into a separate object. Refactoring approach: Intro
duce ParameterObject or Extract Class.

FINAL ANSWER:
Агент зупинився: вичерпано ліміт кроків (2) без фінальної відповіді.

[agent] steps: 2, LLM calls: 2, stop: max_steps, time: 2.80 s
trace saved: traces/trace_20261024_181403.json
```

Модульні тести і результат pytest

```
kate@Lenovo:~/ai-agents/lab01-agents/manual-agent$ uv run pytest -q
...........
11 passed in 0.81s
```

Просунуте завдання: Було написано тести циклу агента без звернень до API з підміною функції рішень сценарієм: завершення за finish, зупинка за MAX_STEPS, перетворення невідомої дії на спостереження.

```
from agent.loop import run_agent


def scripted(decisions):
    queue = iter(decisions)


    def decide(goal, observations, step, max_steps):
        return next(queue)


    return decide


def test_lookup_then_finish():
    decide = scripted([
        {"action": "lookup_code_smell", "arguments": {"name":
"primitive-obsession"},
         "decision_summary": "Потрібні дані про Primitive Obsession."},
        {"action": "finish", "arguments": {"answer": "Primitive
Obsession це надмірне використання примітивних типів"},
         "decision_summary": "Даних достатньо."},
```

```
    ])
    state = run_agent("Що таке Primitive Obsession?", decide=decide)
    assert state.stop_reason == "finish"
    assert state.llm_calls == 2
    assert state.observations[0].startswith("primitive-obsession:")


def test_stops_at_max_steps():
    def endless(goal, observations, step, max_steps):
        return {"action": "lookup_code_smell", "arguments": {"name":
"primitive-obsession"}}


    state = run_agent("Зациклитися", decide=endless, max_steps=3)
    assert state.stop_reason == "max_steps"
    assert state.step == 3


def test_invalid_action_becomes_observation():
    decide = scripted([
        {"action": "rm_rf", "arguments": {}},
        {"action": "finish", "arguments": {"answer": "ok"}},
    ])
    state = run_agent("Тест", decide=decide)
    assert state.observations[0].startswith("ERROR: unknown action")
    assert state.stop_reason == "finish"
```

ADK-агент: пояснення призначення файлів проєкту, інструкція, інструмент, результати чотирьох перевірок, результати lint і тестів

Файл	Що це	У Git?
app/agent.py	Визначення агента: модель, інструкція, інструменти, обгортка App	Так
pyproject.toml	Метадані проєкту і залежності з діапазонами версій	Так
uv.lock	Точні версії всіх залежностей для відтворюваного	Так

Файл	Що це	У Git?
	встановлення	
agents-cli-manifest.yaml	Де лежить код агента (agent_directory) і з якими параметрами створено проєкт	Так
tests/	Модульні, інтеграційні тести і набори оцінювання	Так
GEMINI.md	Інструкції для coding-агентів, які працюватимуть із репозиторієм	Так
.env	Ключ API і параметри Google Cloud	Ні
.venv/	Віртуальне середовище, відтворюється з uv.lock	Ні

```
INSTRUCTION = """

ROLE: An educational agent focused on code smells for programming
students.

PURPOSE: To answer questions about specific code smells based
exclusively on

verified data returned by the `lookup_code_smell` tool.

LANGUAGE: English, unless the user requests a different language.

BEHAVIOR RULES:

1. Before providing information about a specific code smell, call
`lookup_code_smell`

separately for each one mentioned.

2. If the result begins with "NOT_FOUND," explicitly state that no
verified

data is available; do not supplement the answer with your own knowledge.

3. When discussing specific code smells, rely solely on facts derived
from the

tool's output.

4. For general questions (e.g., "What is a code smell?"), provide a
brief

response without invoking the tool.

5. Do not call the tool more than once with the same argument.

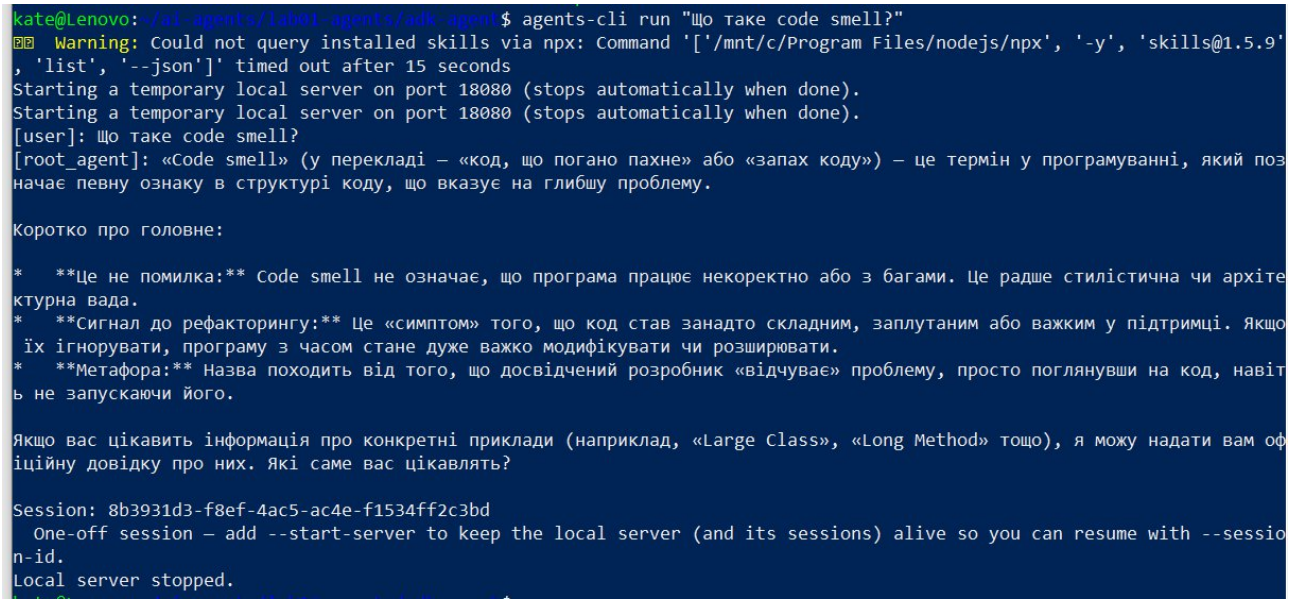
LIMITATIONS: Do not perform actions for which no tool exists; do not
invent sources.

"""
```

Інструменти використовуються такі ж самі, як і в manual-agent.

Обов’язкові перевірки ADK-агента:

№	Запит	Очікувана поведінка
1	Що таке code smell?	Відповідь без виклику інструмента
2	Порівняй Long Method і Duplicated Code	Два виклики lookup_code_smell, відповідь лише з фактів інструмента
3	Опиши code smell Object-Orientation Abusers	Виклик з name="object-orientation-abusers", результат NOT_FOUND, агент визнає відсутність даних
4	agents-cli run "<власний запит>"	Відповідь у терміналі без помилок



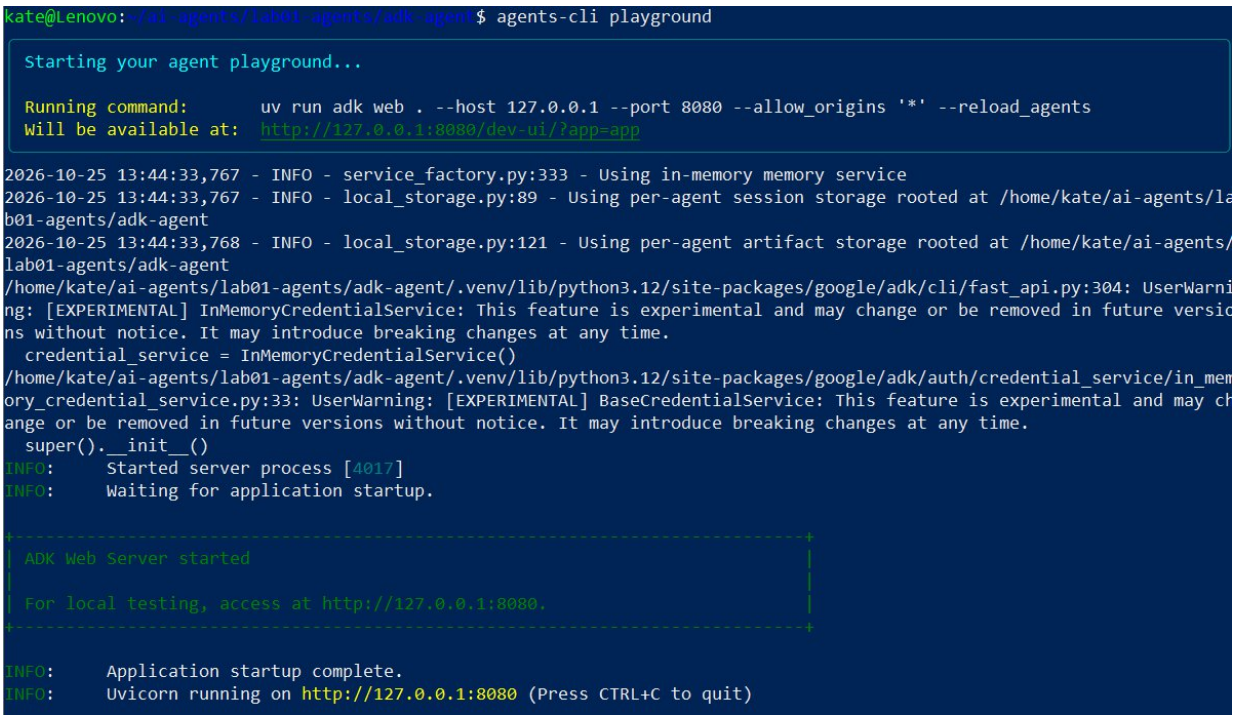


Рисунок – Запуск agents-cli playground

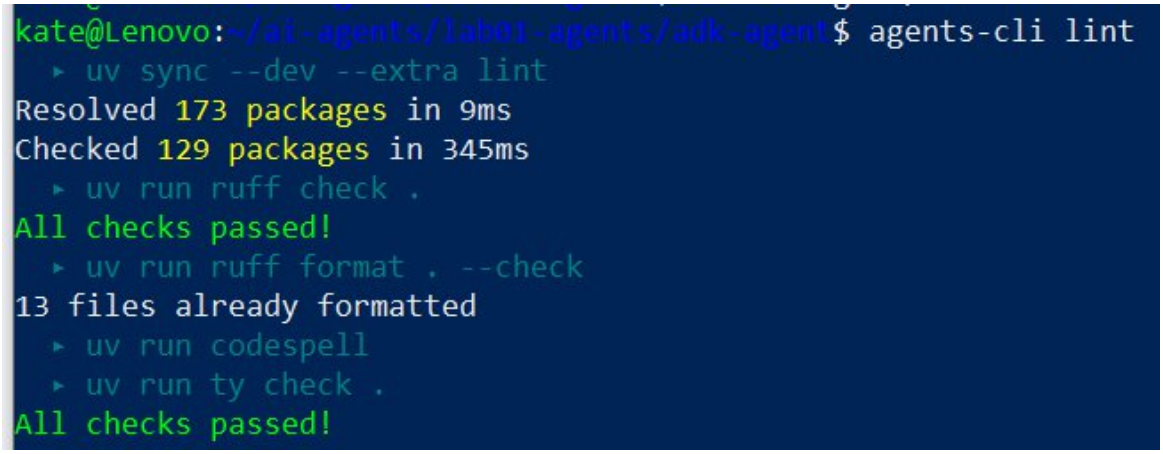


Рисунок – Запуск agents-cli lint

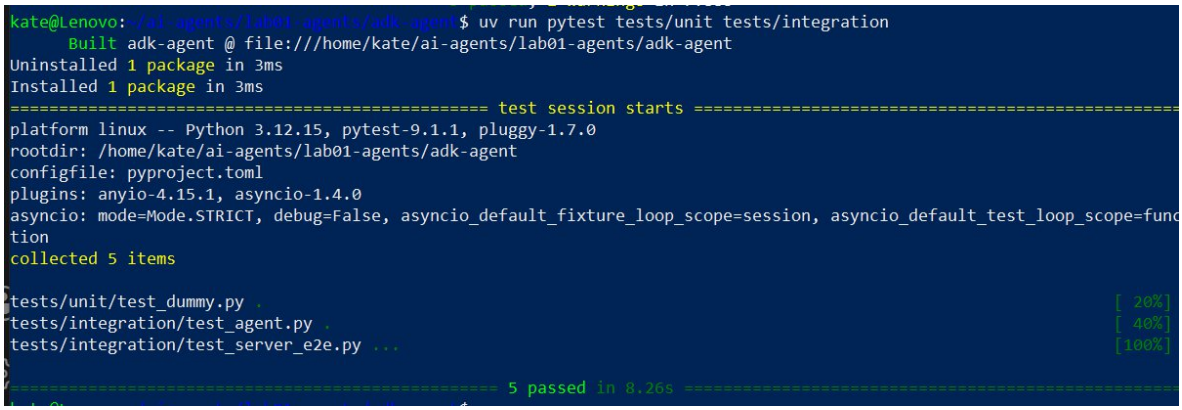


Рисунок – Запуск uv run pytest tests/unit tests/integration

Таблиця відповідності власного агента і ADK

Елемент	Частина А (власний код)	Частина Б (ADK)
Мета	Аргумент main.py	Повідомлення користувача в сесії
Стан	AgentState	Сесія: історія подій і словник state
Спостереження	build_observation()	ADK збирає контекст з інструкції, історії подій і результатів інструментів
Контракт рішення	Власний JSON {action, arguments, decision_summary}	Function call у форматі Gemini API; схему побудовано з сигнатури і docstring
Валідація	validate_decision(), білий список	Виконуються лише функції зі списку tools; перевірку значень аргументів пише розробник у самому інструменті
Диспетчер	execute_action()	ADK викликає Python-функцію за назвою
Умова зупинки	Дія finish або MAX_STEPS	Модель відповідає текстом без виклику інструмента; загальний ліміт викликів моделі задає RunConfig.max_llm_calls
Трасування	traces/*.json	Події сесії в Playground
Обсяг коду агента	≈ 280 рядків у 7 модулях пакета agent	≈ 60 рядків у agent.py

Методика експерименту, 5 запитів, таблиця результатів для трьох систем

Тип	Призначення	Приклад	Очікувана траєкторія агента
A	Один об’єкт	Розкажи про Data Clumps	lookup_code_smell → finish

Тип	Призначення	Приклад	Очікувана траєкторія агента
B	Порівняння двох об’єктів	Порівняй Long Method і Large class	lookup_code_smell × 2 → finish
C	Порівняння трьох об’єктів	Чим відрізняються Long Method, Large class та Duplicated Code?	lookup_code_smell × 3 → finish
D	Об’єкт, якого немає в середовищі	Опиши запах коду Object-Orientation Abusers	lookup_code_smell → NOT_FOUND → finish з визнанням відсутності даних
E	Загальне питання, цикл зайвий	Що таке запах коду?	finish одразу або без звернення

Метрика	Як вимірювати
Правильність	2 - повна і правильна відповідь; 1 - часткова; 0 - хибна. Еталон — дані середовища і джерела, вказані в environment.py
Обґрунтованість	Чи кожне твердження про об’єкти підтверджене спостереженням: так / частково / ні. Для one-shot перевіряють вручну за джерелами
Галюцинація	Чи є у відповіді факт, якого немає в джерелах (особливо для запиту D): так / ні
Виклики LLM	Поле llm_calls у трасуванні; для one-shot — 1; для ADK - за подіями Playground
Дії	Кількість lookup / викликів інструмента
Час, с	Виведення main.py; для ADK — секундомір або час подій Playground

№	Тип	Система	Правильність	Обґрунт.	Галюц.	Виклики LLM	Дії	Час, с
1	A	One-shot	2	так	ні	1	0	41.33
2	A	Manual	2	так	ні	2	1	2.11
3	A	ADK	2	так	ні	2	1	93.52
4	B	One-shot	2	так	ні	1	0	4.97
5	B	Manual	2	так	ні	3	2	8.08
6	B	ADK	2	так	ні	2	2	2.83
7	C	One-shot	2	так	ні	1	0	9.55
8	C	Manual	2	так	ні	4	3	3.79
9	C	ADK	2	так	ні	2	3	2.57
10	D	One-shot	1	так	ні	1	0	24.57
11	D	Manual	2	так	ні	2	1	171.61
12	D	ADK	2	так	ні	1	1	1.45
13	E	One-shot	2	так	ні	1	0	7.37
14	E	Manual	2	так	ні	4	3	6.27
15	E	ADK	2	так	ні	1	0	6.52

Аналіз результатів

Агентний підхід дав найкращий результат для запиту типу D, який передбачав пошук об’єкта, відсутнього в середовищі. One-shot отримав оцінку правильності

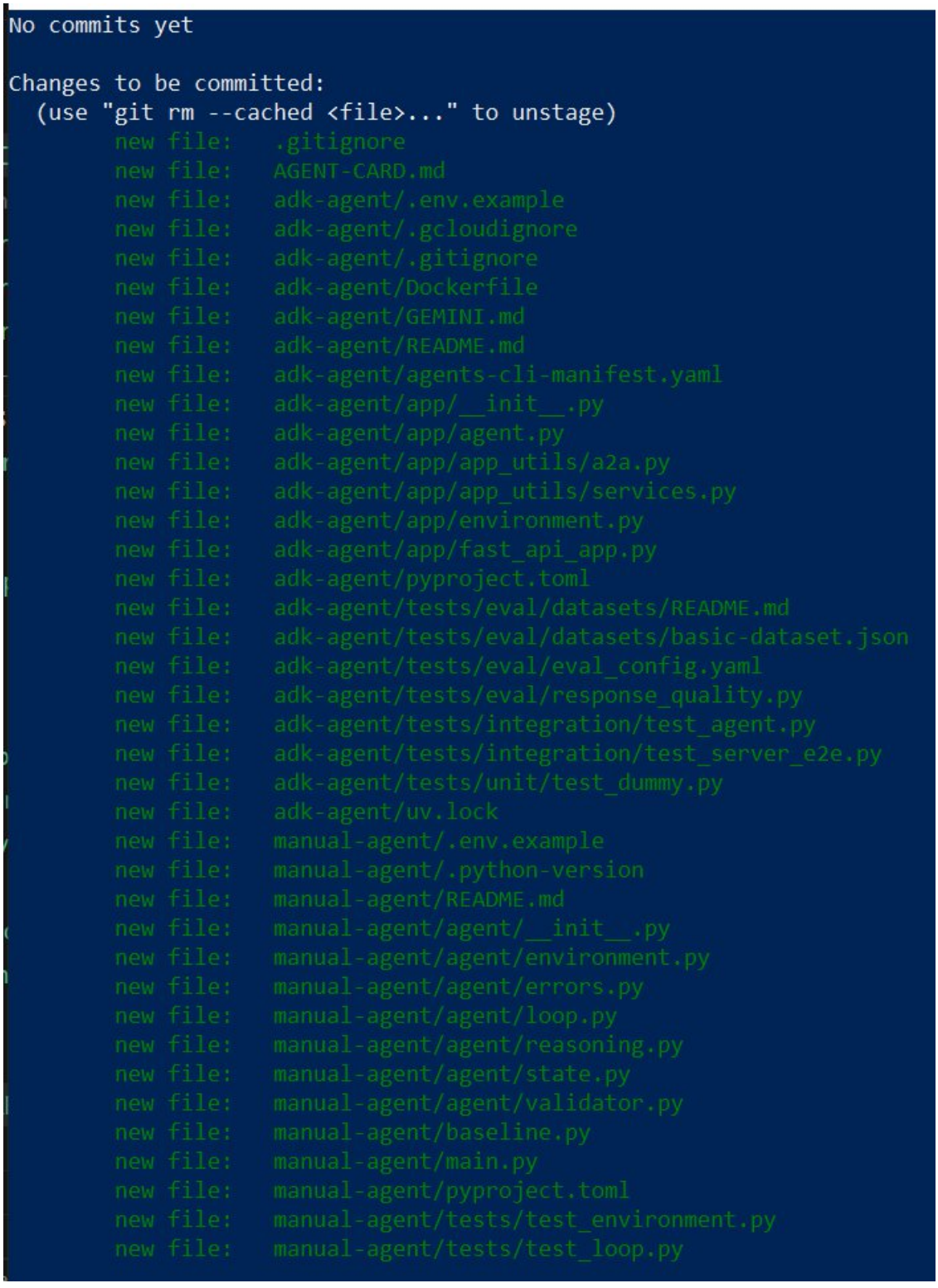
1, тоді як власний агент і ADK-агент - 2, оскільки коректно обробили відсутність даних. Для інших типів запитів усі реалізації отримали максимальну оцінку правильності 2, тому перевага агентів за цим критерієм не була підтверджена. One-shot виявився достатнім для простих запитів, а також для порівняння двох і трьох об’єктів.

Ціна агентності полягає у збільшенні складності реалізації, кількості викликів LLM і потенційних витрат токенів. Власний агент потребував 2-4 викликів LLM, тоді як ADK-агент – 1-2. Затримка суттєво відрізнялася між тестами: власний агент працював від 2,11 до 171,61 с, ADK-агент — від 1,45 до 93,52 с. Отже, агентність не гарантує швидшої відповіді. Кількість використаних токенів у межах експерименту не вимірювалася, тому оцінити відповідні витрати неможливо.

Власний агент потребує додаткової логіки керування циклом Observe-Reason-Act-Evaluate, станом і завершенням роботи. ADK спрощує оркестрацію, але залежить від налаштувань фреймворку. Серед можливих відмов - помилки інструментів, передчасне завершення, повторні виклики та некоректне опрацювання відсутніх даних.

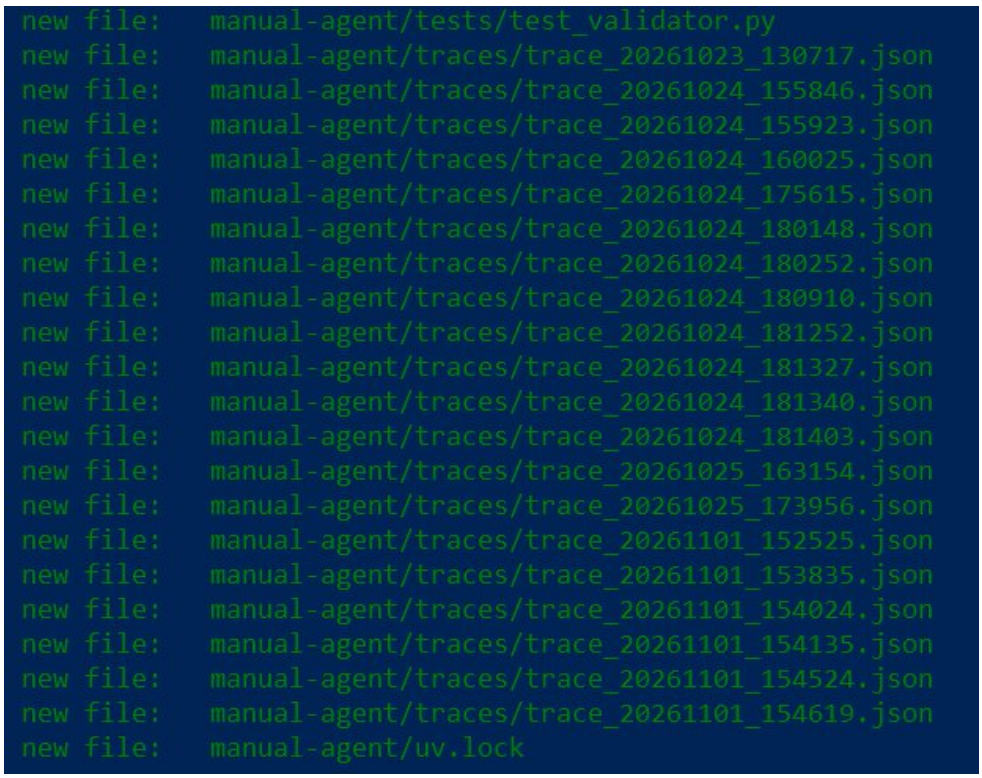
Висновок: one-shot доцільний для простих запитів, а агентний підхід - для завдань, де важливі перевірка інформації та оброблення її відсутності. Отримані результати є попередніми, оскільки кожен тест виконувався один раз. Для об’єктивнішого порівняння необхідно повторити вимірювання та додатково фіксувати використання токенів.

Перед відправкою на гіт



```
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   AGENT-CARD.md
        new file:   adk-agent/.env.example
        new file:   adk-agent/.gcloudignore
        new file:   adk-agent/.gitignore
        new file:   adk-agent/Dockerfile
        new file:   adk-agent/GEMINI.md
        new file:   adk-agent/README.md
        new file:   adk-agent/agents-cli-manifest.yaml
        new file:   adk-agent/app/__init__.py
        new file:   adk-agent/app/agent.py
        new file:   adk-agent/app/app_utils/a2a.py
        new file:   adk-agent/app/app_utils/services.py
        new file:   adk-agent/app/environment.py
        new file:   adk-agent/app/fast_api_app.py
        new file:   adk-agent/pyproject.toml
        new file:   adk-agent/tests/eval/datasets/README.md
        new file:   adk-agent/tests/eval/datasets/basic-dataset.json
        new file:   adk-agent/tests/eval/eval_config.yaml
        new file:   adk-agent/tests/eval/response_quality.py
        new file:   adk-agent/tests/integration/test_agent.py
        new file:   adk-agent/tests/integration/test_server_e2e.py
        new file:   adk-agent/tests/unit/test_dummy.py
        new file:   adk-agent/uv.lock
        new file:   manual-agent/.env.example
        new file:   manual-agent/.python-version
        new file:   manual-agent/README.md
        new file:   manual-agent/agent/__init__.py
        new file:   manual-agent/agent/environment.py
        new file:   manual-agent/agent/errors.py
        new file:   manual-agent/agent/loop.py
        new file:   manual-agent/agent/reasoning.py
        new file:   manual-agent/agent/state.py
        new file:   manual-agent/agent/validator.py
        new file:   manual-agent/baseline.py
        new file:   manual-agent/main.py
        new file:   manual-agent/pyproject.toml
        new file:   manual-agent/tests/test_environment.py
        new file:   manual-agent/tests/test_loop.py
```



Що саме перетворює програму, яка викликає LLM API, на AI-агента?

Те, що ми використовуємо LLM API не робить програму AI-агентом. Звичайна програма може надсилати запит до мовної моделі, отримувати відповідь і повертати її користувачу. AI-агент здатний самостійно обирати наступні кроки для досягнення визначеної мети, взаємодіяти із середовищем і коригувати свою поведінку на основі отриманих результатів.

Основними складовими агента є мета, стан, спостереження, політика та дія. Мета визначає бажаний результат роботи. Стан містить інформацію про поточний прогрес, історію взаємодії та доступні дані. Спостереження - інформація, яку агент отримує від користувача або середовища. Політика визначає, яку дію потрібно виконати на основі поточного стану та спостережень.

Важливою характеристикою агента є цикл агента: отримання спостереження, аналіз ситуації, вибір і виконання дії, оцінювання результату та оновлення стану. Цей процес повторюється, доки не буде досягнуто мети або виконано умову зупинки. Середовище - це зовнішній світ, з яким взаємодіє агент: користувач,

API, база даних або набір інструментів. Фреймворк допомагає реалізувати цю взаємодію, керувати станом, викликами інструментів і циклом виконання.

Отже, головна відмінність AI-агента від звичайної програми з LLM API полягає в наявності керованого циклу прийняття рішень і виконання дій для досягнення мети, а не лише в генерації тексту у відповідь на запит.

Висновки

Під час виконання лабораторної роботи налаштували відтворюване середовище розроблення AI-агентів; побудували на Python власного мінімального агента з циклом Observe, Reason, Act, Evaluate без agent framework; відтворили того самого агента на Google ADK з власним інструментом; експериментально порівняли одиничний виклик LLM, власного агента і ADK-агента на п’яти типах запитів; оформили паспорт агента. Власний цикл агента я обрала би тоді, коли логіка роботи є відносно простою, потрібен повний контроль над послідовністю дій, станом і умовами зупинки, а також важливо мінімізувати залежності від сторонніх бібліотек. Фреймворк доцільніше використовувати для складніших агентів, які працюють із багатьма інструментами, потребують керування пам’яттю, обробки помилок і взаємодії з різними сервісами.